SUNUM 2

YAZILIM SİSTEMLERİNDE KARMAŞIKLIK ANALİZ PLATFORMU

Salih Sırrı Doğan: Scrum Master & Backend
Halil Taşar: Metrik Algoritmalar
Çağrı Demir: Veritabanı & API
Mustafa Çoban: Frontend & UI
Fatma Serra Şimşek: Bağımlılık Haritalama

BACKLOG KALİTESİ

SPRİNT HEDEFLERİ

01

SPRİNT-1

Araştırma ve Planlama

- Projenin teknik altyapısını belirlemek için kapsamlı bir araştırma süreci yürütmek (literatür taraması)
- Kullanılacak teknolojileri, algoritmaları ve mimari yaklaşımları belirleyerek projenin yol haritasını oluşturmak

SPRİNT-2

Çekirdek Analiz Motoru

- Araştırma sonuçlarını uygulamaya dökerek projenin çekirdek analiz yeteneklerini geliştirmek
- Projelerindeki bağımlılık ilişkilerini çıkaran, döngüsel bağımlılıkları tespit eden, kod karmaşıklığını ölçen algoritmaların yazılması

SPRİNT-3

Geliştirme ve Görselleştirme

- Analiz motorunu geliştirerek çalışan bir MVP ortaya koymak
- Kullanıcı dostu bir web arayüzü inşa etmek
- Analiz sonuçlarını görsel harita olarak sunmak ve genel proje sağlık skoru üretmek

SPRİNT-4

Entegrasyon

- Sistemin analiz kapasitesini farklı dilleri kapsayacak şekilde genişletmek
- Analiz sonuçlarını dışa aktarılabilir raporlara dönüştürmek
- Tüm modüllerin entegrasyon testlerini tamamlamak ve projeyi teslime hazır bir ürün haline getirmek

USER STORİES & KABUL KRİTERLERİ

02

USER STORY-1

Yazılım geliştiricisi olarak, bir projedeki tüm dosyalar arası import ilişkilerini otomatik olarak taratmak istiyorum, böylece hangi dosyanın hangi dosyaya/kütüphaneye bağımlı olduğunu manuel inceleme yapmadan görebilirim.

USER STORY-2

Bir yazılım mimarı olarak, projede döngüsel bağımlılıkların olup olmadığını otomatik olarak tespit ettirmek istiyorum. Böylece oluşabilecek mimari hataları erken aşamada fark edebilirim.

USER STORY-3

Bir proje yöneticisi olarak projedeki bağımlılık ilişkilerini ve karmaşıklık seviyesini anlaşılır bir görsel harita şeklinde görmek ve sonuçları dosya olarak dışarı aktarabilmek istiyorum. Böylece proje mimarisini hızlıca değerlendirebilir, ekip sunumlarında kullanabilir ve geçmiş analizlerle karşılaştırabilirim.

KABUL KRİTERİ-1

Sistem kullanıcının seçtiği klasördeki tüm dosyaları tarar ve her dosyadaki import X ifadeleri doğru şekilde çıkarılır. Her dosya için bağımlılık listesi (kaynak_dosya, hedef_modül) formatında döner

KABUL KRİTERİ-2

Bağımlılık grafiğindeki tüm kapalı döngüleri tespit edilir. Her döngü zincir formatında kullanıcıya gösterilir. Döngü yoksa kullanıcıya "döngü bulunamadı" bilgisi verilir. Birden fazla döngü varsa hepsi ayrı ayrı listelenir.

KABUL KRİTERİ-3

Sistem bağımlılıkları yönlü grafik olarak çizer. Analiz sonuçları PDF formatında dışa aktarılabilir. Dışa aktarılan raporda bağımlılık listesi, döngü raporu ve karmaşıklık tablosu eksiksiz yer alır. Görsel bağımlılık haritası rapora görüntü olarak yer alır.

USER STORİES & KABUL KRİTERLERİ

03

USER STORY-4

Yazılım geliştiricisi olarak bir projedeki fonksiyonların karmaşıklık düzeyini ölçtürebilmeliyim. Bu sayede hangi fonksiyonların refactor edilmesi gerektiğini somut verilere dayanarak belirleyebilirim.

USER STORY-5

Bir takım lideri olarak bağımlılık analizi ve karmaşıklık analizi sonuçlarının birleştirilip projeye genel bir skor verilmesini istiyorum, böylece projenin genel kalite durumunu tek bir metrikle takip edebilir ve zaman içindeki değişimi ölçebilirim.

USER STORY-6

Son kullanıcı olarak analiz edilecek projeyi bilgisayarımdan dosya yükleyerek ya da GitHub repo linki yapıştırarak sisteme aktarabilmeliyim.

KABUL KRİTERİ-4

Her fonksiyon için cyclomatic complexity değeri hesaplanır. Karmaşıklık eşik değerini aşan fonksiyonlar belirlenir. Dosya bazında ortalama karmaşıklık skoru raporlanır ve kullanıcıya sunulur.

KABUL KRİTERİ-5

Sistem bağımlılık ve karmaşıklık analizlerinin sonuçlarını birleştirerek genel bir skor hesaplar. Skoru etkileyen alt metrikler (döngü sayısı, toplam bağımlılık sayısı vb.) detaylı olarak gösterilir.

KABUL KRİTERİ-6

Sistemde hem dosya yükleme alanı bulunur, kullanıcı kaynak dosyalarını veya zip arşivini yükleyebilir. Hem de URL giriş alanı bulunur, kullanıcı GitHub vb. repo linkini yapıştırarak analizi başlatabilir

NON-FUNCTİONAL REQUİREMENTS (NFR)

04

1.GÜVENLİK VE GİZLİLİK

Sistem, analiz edilmek üzere çekilen kaynak kodları hiçbir şekilde kalıcı veritabanında saklamamalı, analiz bitiminde bellekten güvenli bir şekilde silmelidir.

2.PERFORMANS

Analiz başlatıldığında kullanıcıya ilerleme göstergesi sunulmalı ve dosya okuma, analiz, görselleştirme işlemleri 50 saniyeyi aşmamalıdır.

3.KULLANILABİLİRLİK

Sistem arayüzü sade ve fonksiyonel olmalıdır. Kod kalitesini renk kodlarıyla (Yeşil: Güvenli, Sarı: Uyarı, Kırmızı: Kritik) görselleştirerek kullanıcının sistemin mevcut durumunu kolayca kavramasını sağlamalıdır.

4.HATA TOLERANSI

Bozuk dosyalar, boş klasörler veya erişim izni olmayan dosyalar programı çökertmemelidir. Hatalı girdiler verildiğinde program uyarı mesajıyla devam etmelidir.

GÖREV DAĞILIMLARI

BU HAFTA NELER YAPILDI?

05

SALİH SIRRI DOĞAN

- Analiz akışını yöneten backend orchestrator yapısı geliştirildi.
- Modüller arası veri formatı ve iletişim yapısı belirlendi.
- API endpointleri ve veri akışı tasarlandı.
- Sistem mimarisi ve entegrasyon planı oluşturuldu.

MUSTAFA ÇOBAN

- Streamlit tabanlı dinamik dashboard arayüzü geliştirildi.
- User Story gereksinimlerini karşılayan 4 ana analiz sekmesi modüler olarak tasarlandı

HALİL TAŞAR

- McCabe, Halstead ve literatüre yeni eklediğimiz ICC metriklerini hesaplayan çekirdek analiz motoru geliştirildi.
- Mock data ile test edildi.

FATMA SERRA ŞİMŞEK

- Python AST ile import ayrıştırma ve NetworkX yönlü grafik modeli geliştirildi.
- NetworkX kütüphanesi ile döngüsel bağımlılıkları tespit eden fonksiyon yazıldı.

ÇAĞRI DEMİR

- FastAPI kullanarak backend API geliştirildi.
- SQLAlchemy ile PostgreSQL (Supabase) veritabanı bağlantısı kuruldu.
- Proje oluşturma (POST /api/projects) endpoint'i geliştirildi ve çalışır hale getirildi.
- API üzerinden gelen verilerin doğrulanması ve veritabanına kaydedilmesi sağlandı.

TEŞEKKÜRLER